Week 1 Lab 3 — Matplotlib Refresher

28th October 2026

Introduction

Matplotlib is a 2D and 3D graphics library for generating scientific figures.

With Matplotlib you can:

- get started with simple plots
- You can customise labels and texts for your plots
- Great control of every element in a figure
- High-quality output in many formats
- Customizable

Matplotlib allows you to create reproducible figures programmatically. Let's learn how to use it! Please explore the official Matplotlib web page: http://matplotlib.org/

[175]:
```
# Importing matplotlib needs .pyplot. Common mistake - students forget to␣
↪attach .pyplot

import matplotlib.pyplot as plt
```

1. Matplotlib Basics

We will begin using two numpy arrays to plot. In your ML programs mostly you will be passing numpy arrays or pandas columns (which essentially also behave like arrays).

[176]:
```
import numpy as np
```

[177]:
```
x = np.arange(2,12)
```

[178]:
```
y = 1.5*x
```

[179]:
```
x
```

[179]:
```
array([ 2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

[180]:
```
y
```

[180]:
```
array([ 3. ,  4.5,  6. ,  7.5,  9. , 10.5, 12. , 13.5, 15. , 16.5])
```

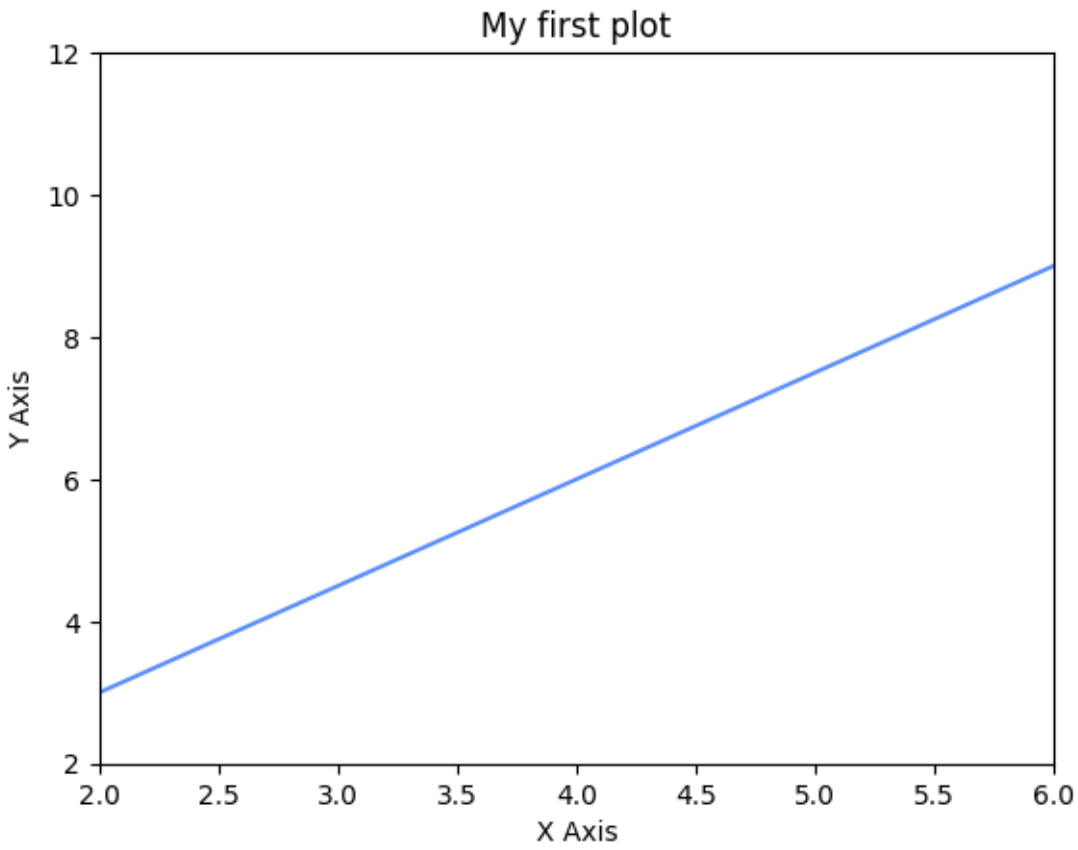
Use Matplotlib with plt.plot() functions

Lets create a very simple line plot using the following

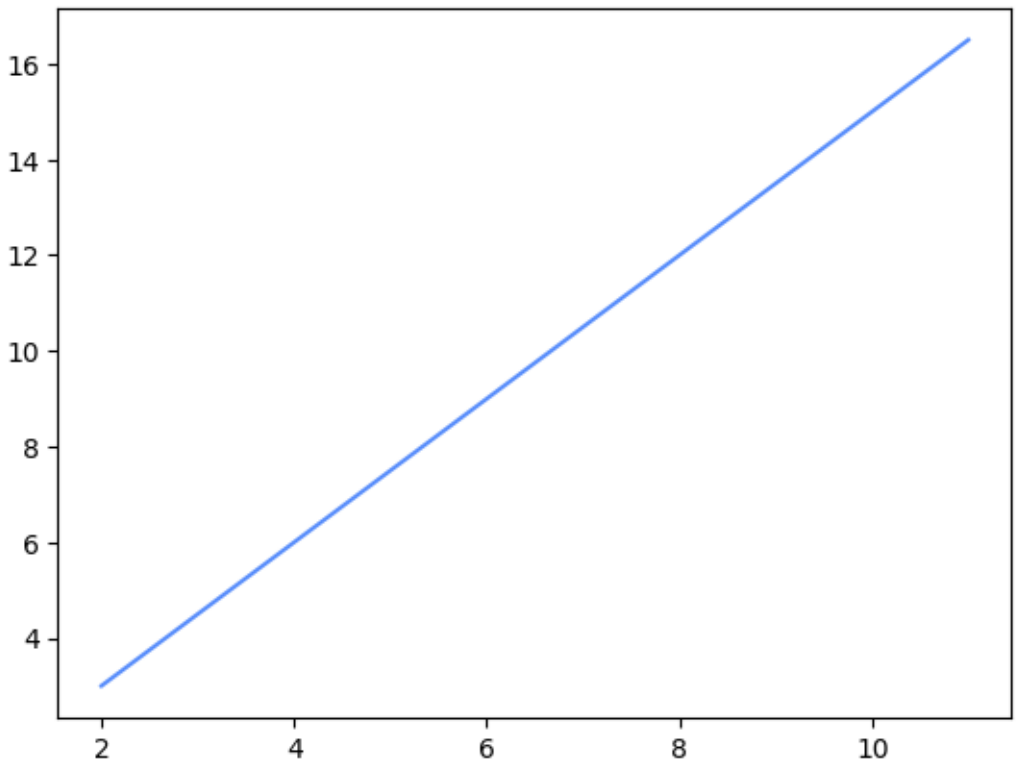
[181]:
```
plt.plot(x, y)
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.title('My first plot')
plt.show() #show() is not absoutely necessary for jupyter notebook, but with .
 ↪py scripts you will need this
#command to see the graph/plot
```



[182]:
```
plt.plot(x, y)
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.title('My first plot')
plt.xlim(2,6) # Lower Limit, Upper Limit
plt.ylim(2,12) # Lower Limit, Upper Limit
plt.show()
```



[183]:
```
plt.plot(x,y)
plt.savefig('./images/week_1/plot1.png')
```



Customisation

You can customise the plots using some other arguments to the plot call. Visit https://matplotlib.org/ site and see if you can bring in some customisation to the above plot. Share it with others in the class.

Sources used:

- Matplotlib line plot reference
- Matplotlib named colours
- Matplotlib Linestyles



[184]:
```
fig, ax = plt.subplots()
ax.plot(x, y, linestyle='--', linewidth=2, color='green', marker='o',␣
 ↪markersize=10, markerfacecolor='deepskyblue', markeredgewidth=2,␣
 ↪markeredgecolor='fuchsia')

#ax.set(xlabel='time (s)', ylabel='voltage (mV)', title='The original plot with␣
 ↪some customisation')

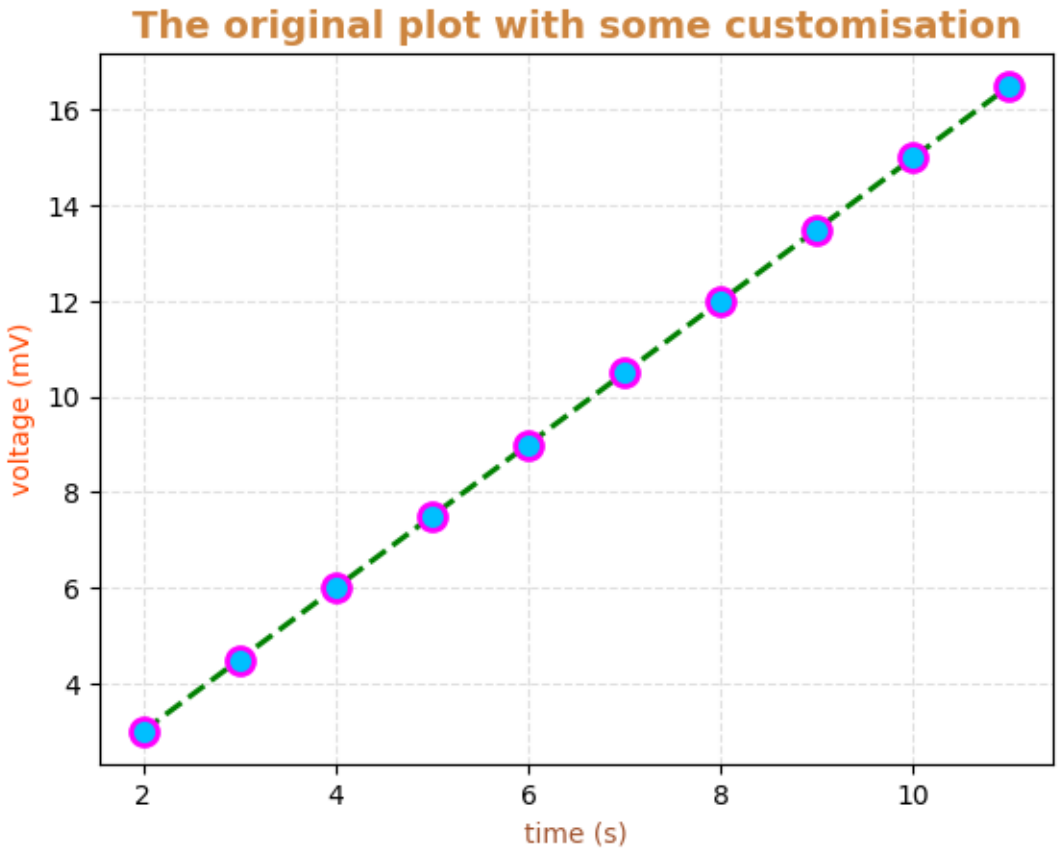
ax.set_title(
    'The original plot with some customisation',
    color='peru',
    fontweight='bold',
    fontsize=14,
)
ax.set_xlabel('time (s)', color='sienna')
```

```
ax.set_ylabel('voltage (mV)', color='orangered')

# Create a subtle, light gray dashed grid line behind the data
ax.grid(
    visible=True,           # Ensure grid is visible
    color='lightgray',      # Line color
    linestyle='--',         # Dashed line style
    linewidth=0.8,          # Line thickness
    alpha=0.7               # Transparency
)

# Keep the grid lines behind your plotted data lines/bars
ax.set_axisbelow(True)

fig.savefig('./images/week_1/plot1_customised.png', bbox_inches='tight')
plt.show()
```



2. Creating figure in Matplotlib
   With this library you can create figure objects and then just call methods or attributes of that object. This approach could help when you have multiple plots on it and treating them

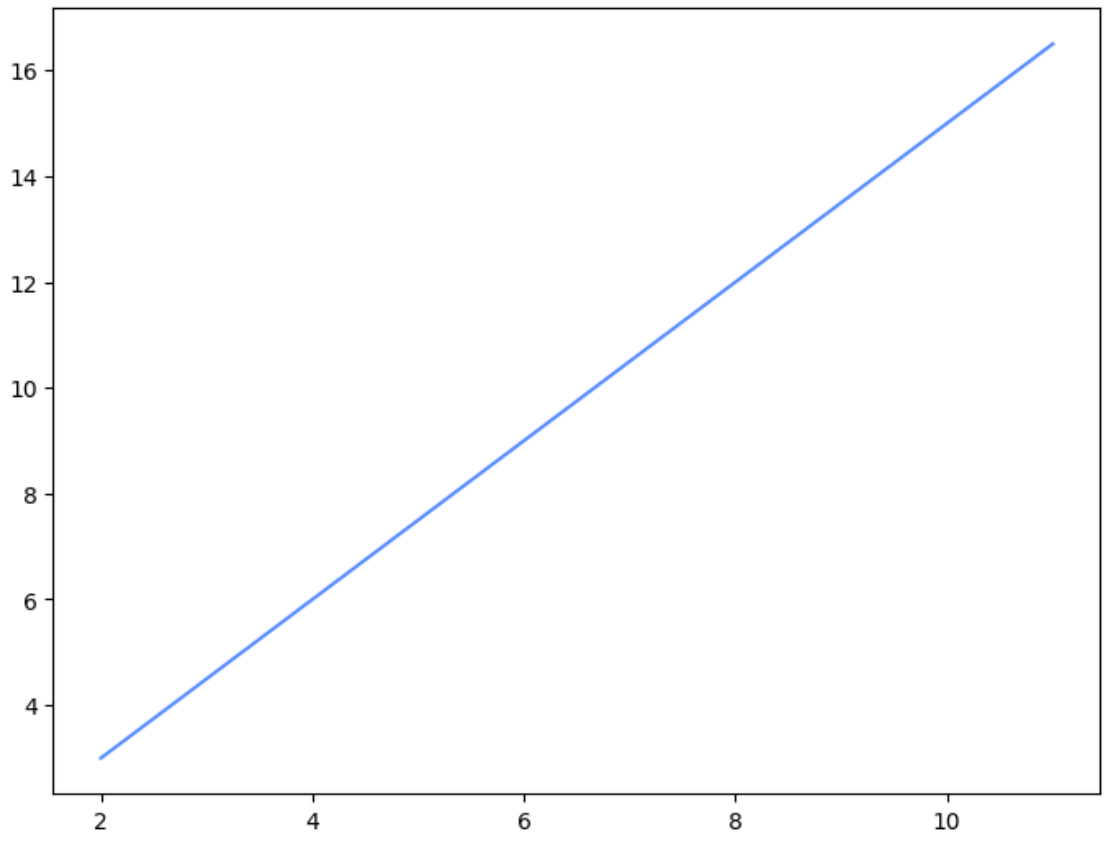
as individual object provides more flexibility.

[185]:
```
# Create Figure (empty canvas)
fig = plt.figure()

# Add set of axes to figure, practice this by changing the left, bottom, width␣
 ↪and height to some other
#numbers in the range of 0 and 1
axes = fig.add_axes([0, 0, 1, 1]) # left, bottom, width, height (range 0 to 1)

# Plot on that set of axes
axes.plot(x, y)

plt.show()
```



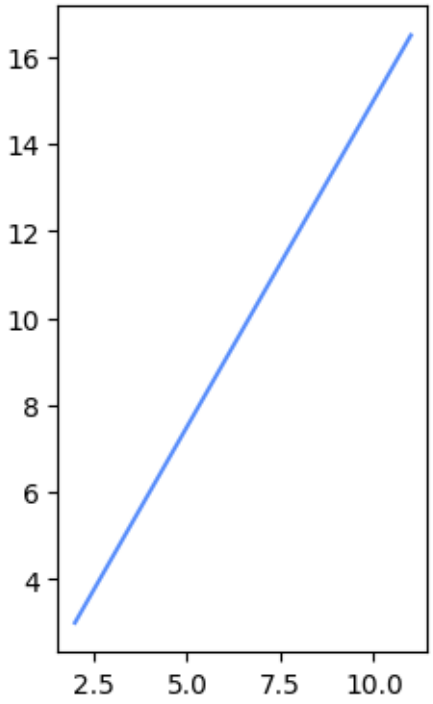
My own code changing the left, bottom, width and height to some other numbers in the range of 0 and 1.

[186]:
```
# Create Figure (empty canvas)
fig = plt.figure()
```

```
#numbers in the range of 0 and 1
axes = fig.add_axes([0.15, 0.15, 0.3, 0.7]) # left, bottom, width, height␣
 ↪(range 0 to 1)

# Plot on that set of axes
axes.plot(x, y)

plt.show()
```



0.0.1 Show the full figure without automatic cropping

By default, this notebook's inline backend uses `bbox_inches='tight'`, which crops the displayed image around the plot and its labels. This can hide changes to the `left` and `bottom` positions in `fig.add_axes([left, bottom, width, height])` because the surrounding empty space is removed.

The command below sets `bbox_inches` to `None`, disabling that automatic cropping and preserving the full figure canvas. The light-gray figure background makes the margins visible, so you can see the plot move when you change `left` or `bottom`. This changes how figures are displayed, not the data or axis limits.

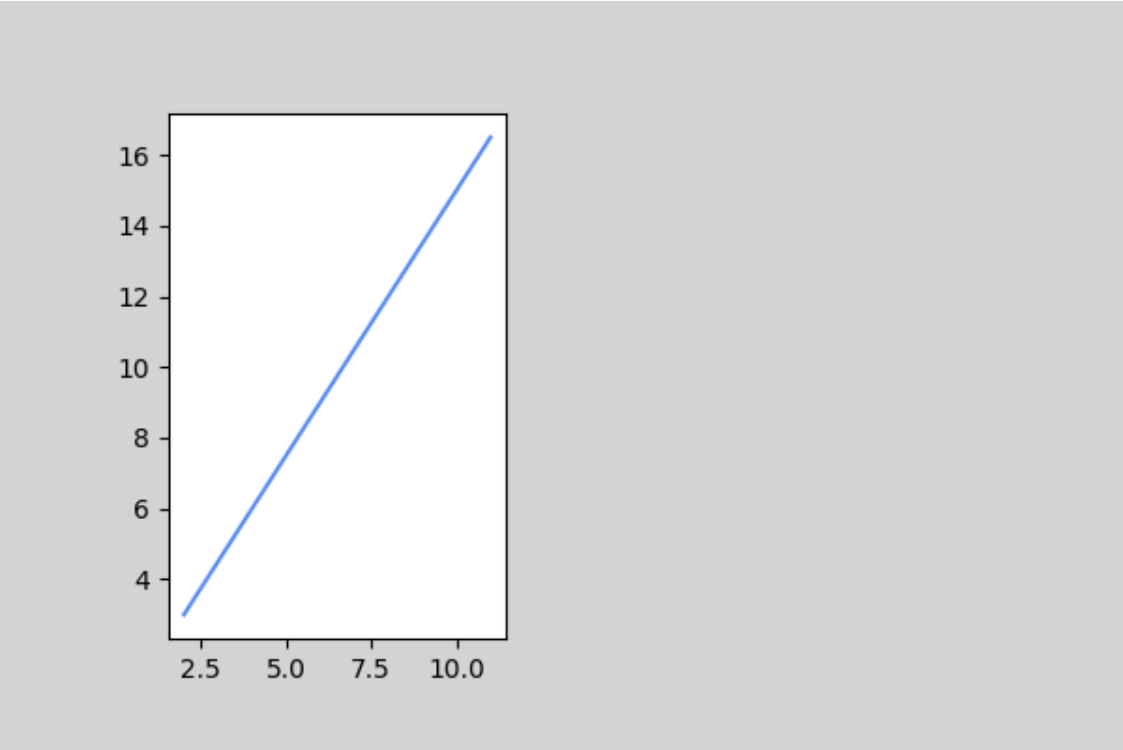
The setting applies to subsequent inline figure displays in the current kernel session. To restore automatic cropping, run:

```
%config InlineBackend.print_figure_kwargs = {'bbox_inches': 'tight'}
```

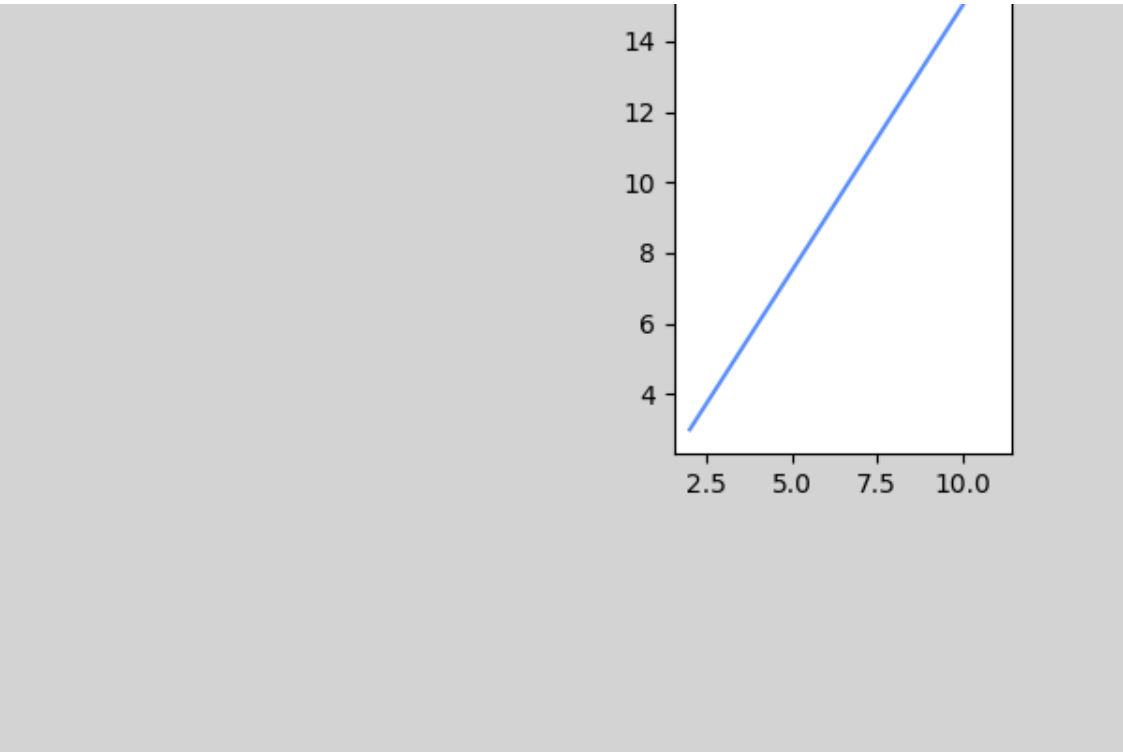
With cropping disabled, keep `left + width <= 1` and `bottom + height <= 1` to keep the plotting area inside the canvas; leave additional room for tick labels and titles.

[187]:
```
%config InlineBackend.print_figure_kwargs = {'bbox_inches': None}

fig = plt.figure(figsize=(6, 4), facecolor='lightgray')
axes = fig.add_axes([0.15, 0.15, 0.3, 0.7])
axes.plot(x, y)
plt.show()
```



[188]:
```
fig = plt.figure(figsize=(6, 4), facecolor='lightgray')
axes = fig.add_axes([0.6, 0.4, 0.3, 0.7])
axes.plot(x, y)
plt.show()
```



[189]:
```
# Restore automatic cropping for subsequent figures
%config InlineBackend.print_figure_kwargs = {'bbox_inches': 'tight'}
```

[190]:
```
import numpy as np
a = np.linspace(0,10,11)
b = a ** 2
```

[191]:
```
a
```

[191]:
```
array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

[192]:
```
b
```

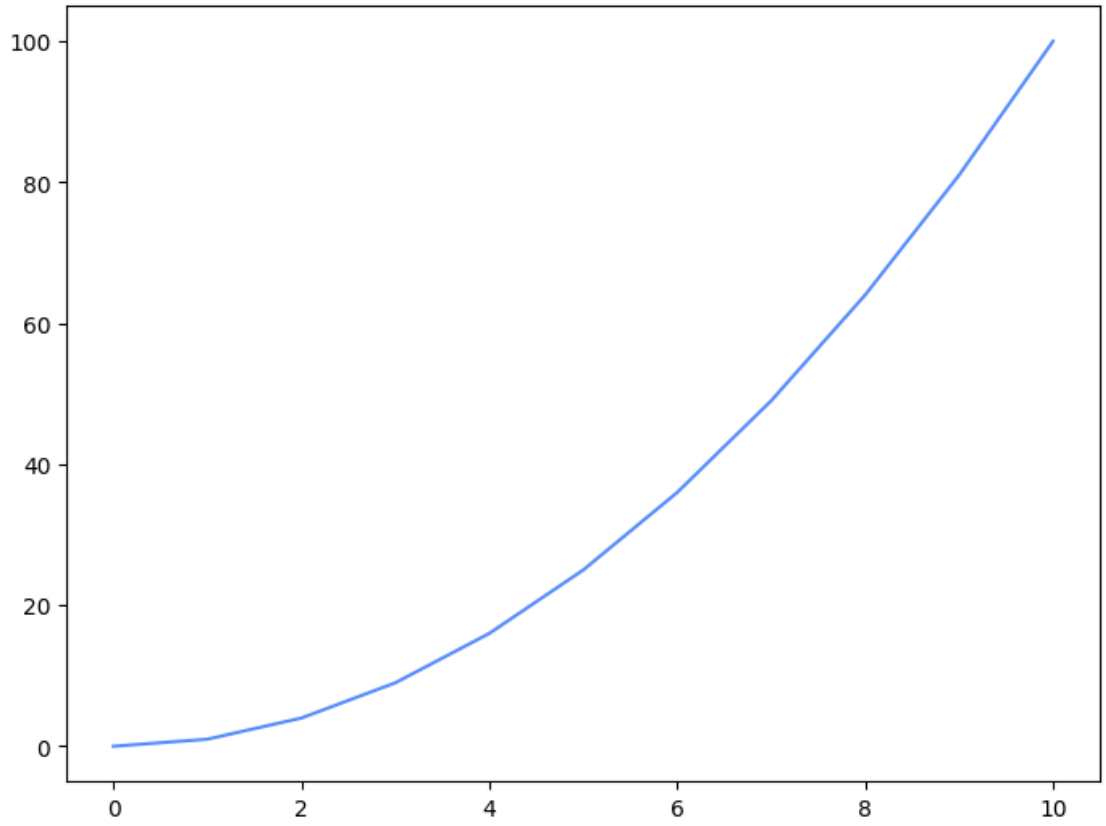
[192]:
```
array([  0.,   1.,   4.,   9.,  16.,  25.,  36.,  49.,  64.,  81., 100.])
```

[193]:
```
# Create Figure (empty canvas)
fig = plt.figure()

# Add set of axes to figure, practice this by changing the left, bottom, width␣
 ↪and height to some other
#numbers in the range of 0 and 1
axes = fig.add_axes([0, 0, 1, 1]) # left, bottom, width, height (range 0 to 1)
```

```
# Plot on that set of axes
axes.plot(a, b)

plt.show()
```



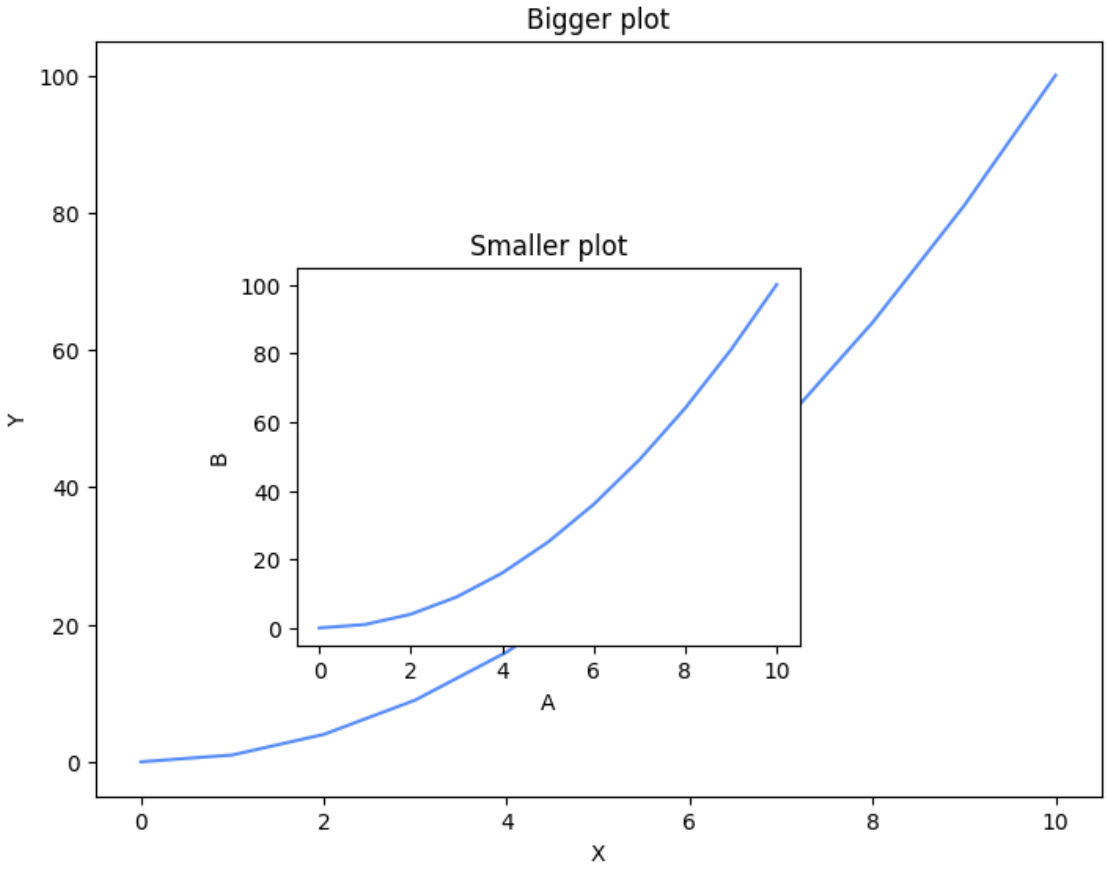
[194]:
```
# Create a blank canvas for figure
fig = plt.figure()

#now add two axes one for bigger and other for smaller plot/figure
axes1 = fig.add_axes([0, 0, 1, 1]) # Bigger figure
axes2 = fig.add_axes([0.2, 0.2, 0.5, 0.5]) # Smaller figure

# Bigger Figure Axes 1
axes1.plot(a, b)

# Use set_ to add to the axes figure
axes1.set_xlabel('X')
axes1.set_ylabel('Y')
axes1.set_title('Bigger plot')
```

```
# Plot Figure Axes 2
axes2.plot(a,b)
axes2.set_xlabel('A')
axes2.set_ylabel('B')
axes2.set_title('Smaller plot');
```



[195]:
```
# bbox_inches ='tight' automatically makes sure the bounding box is correct
fig.savefig('./images/week_1/figure.png',bbox_inches='tight')
```

Practice

Practice further with three plots of your choice and research on matplotlib further to change the parameters of each plot to make it look different.

Sources:

- Matplotlib Parametric curve

First we plot our target curve prior to adding the projections. The curve is defined parametrically

by the following equations:

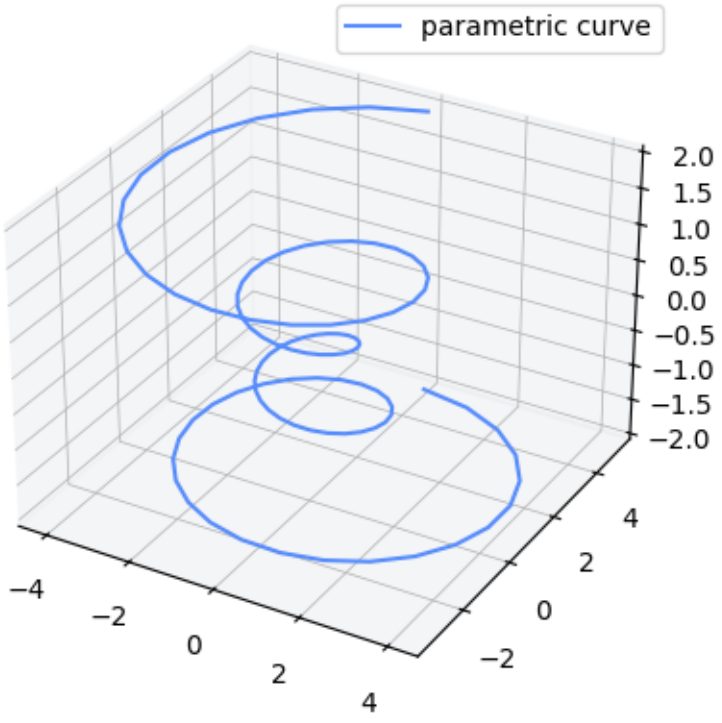
$$x = r\sin(\theta) \tag{1}$$
$$y = r\cos(\theta) \tag{2}$$
$$z = z \tag{3}$$

[196]:
```
ax = plt.figure().add_subplot(projection='3d')

# Prepare arrays x, y, z
theta = np.linspace(-4 * np.pi, 4 * np.pi, 100)
z = np.linspace(-2, 2, 100)
r = z**2 + 1
x = r * np.sin(theta)
y = r * np.cos(theta)

ax.plot(x, y, z, label='parametric curve')
ax.legend()

plt.show()
```



The code below plots the parametric 2D projections of the curve onto the XY and YZ planes. The 2D plots are created using subplots, which allows for multiple plots to be displayed in a single

figure. The aspect ratio of each subplot is set to 'equal' to ensure that the scales of the axes are the same, and a grid is added for better visualization.

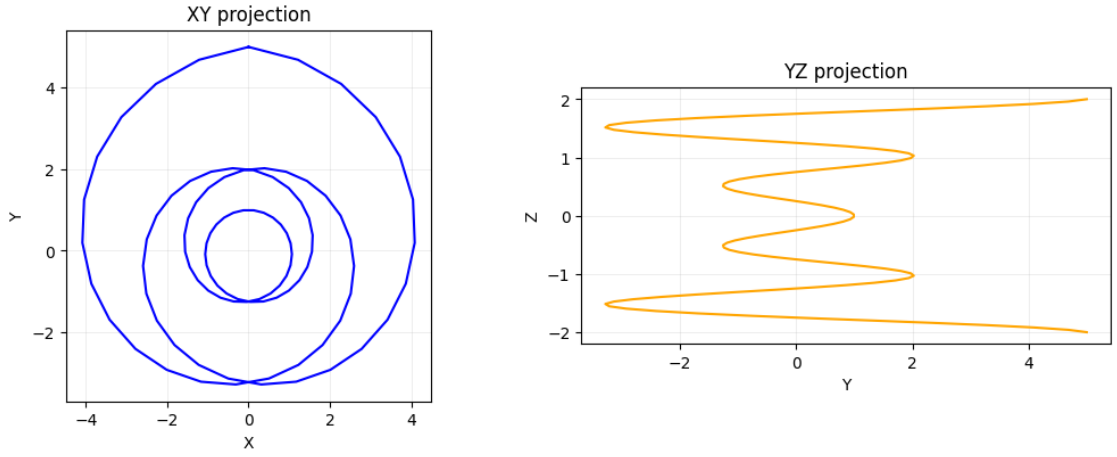
[197]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4), layout='constrained')

axes[0].plot(x, y, color='blue')
axes[0].set(xlabel='X', ylabel='Y', title='XY projection')

axes[1].plot(y, z, color='orange')
axes[1].set(xlabel='Y', ylabel='Z', title='YZ projection')

for ax in axes:
    ax.set_aspect('equal', adjustable='box')
    ax.grid(alpha=0.3)

plt.show()
```



Finally we plot the parametric curve in 3D and overlay the 2D projections onto the main plot. The 3D plot is created using the `projection='3d'` argument in `add_axes`, and the 2D plots are added as smaller axes on top of the main plot. The `zorder` parameter is used to control the layering of the plots, ensuring that the 2D projections are displayed above the 3D plot.

[198]:
```
fig = plt.figure(figsize=(9, 7))

# Prepare arrays x, y, z
theta = np.linspace(-4 * np.pi, 4 * np.pi, 100)
z = np.linspace(-2, 2, 100)
r = z**2 + 1
x = r * np.sin(theta)
y = r * np.cos(theta)

# Main 3D plot
```

```
axes1 = fig.add_axes([0.05, 0.05, 0.85, 0.85], projection='3d')
axes1.plot(x, y, z) # The label is redundant: label='Parametric curve'
axes1.set(
    xlabel='X', ylabel='Y', zlabel='Z',
    #title='Parametric curve'
)
axes1.set_title(
    'Parametric curve',
    color='crimson',
    fontweight='bold',
    fontsize=14,
)
#axes1.legend(loc='lower left')
axes1.xaxis.label.set_color('orangered')
axes1.yaxis.label.set_color('orangered')
axes1.zaxis.label.set_color('orangered')

# Smaller 2D plot overlaid on the main plot
axes2 = fig.add_axes([0.65, 0.65, 0.28, 0.28], zorder=10)
axes2.set_facecolor('white')
axes2.plot(x, y, color='blue')
axes2.set(xlabel='X', ylabel='Y') #, title='XY projection')
axes2.set_title(
    'XY projection',
    color='blue',
    fontweight='normal',
    fontsize=10,
)
axes2.set_aspect('equal', adjustable='box')
axes2.grid(alpha=0.3)
axes2.tick_params(axis='both', labelcolor='blue')
axes2.xaxis.label.set_color('blue')
axes2.yaxis.label.set_color('blue')


# Smaller 2D plot overlaid on the main plot
axes3 = fig.add_axes([0.0, 0.15, 0.28, 0.28], zorder=10)
axes3.set_facecolor('white')
axes3.plot(y, z, color='orange')
axes3.set(xlabel='Y', ylabel='Z') #, title='YZ projection')
axes3.set_title(
    'YZ projection',
    color='orange',
    fontweight='normal',
    fontsize=10
)
axes3.set_aspect('equal', adjustable='box')
```

```
axes3.grid(alpha=0.3)
axes3.tick_params(axis='both', labelcolor='orange')
axes3.xaxis.label.set_color('orange')
axes3.yaxis.label.set_color('orange')

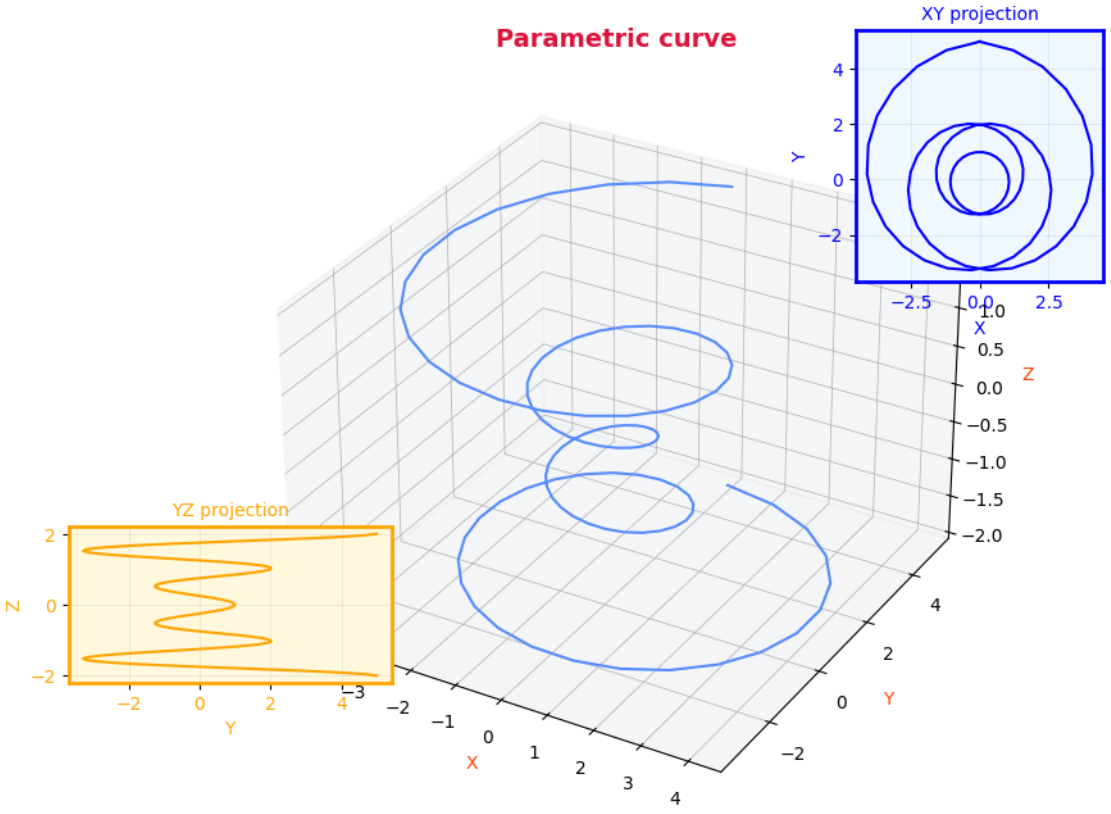
# XY projection border
for spine in axes2.spines.values():
    spine.set_edgecolor('blue')
    spine.set_linewidth(2)

# YZ projection border
for spine in axes3.spines.values():
    spine.set_edgecolor('orange')
    spine.set_linewidth(2)

axes2.set_facecolor('aliceblue')
axes3.set_facecolor('cornsilk')

plt.show()

fig.savefig('./images/week_1/figure_parametric.png',bbox_inches='tight')
```



Creating Sub Plots in Matplotlib

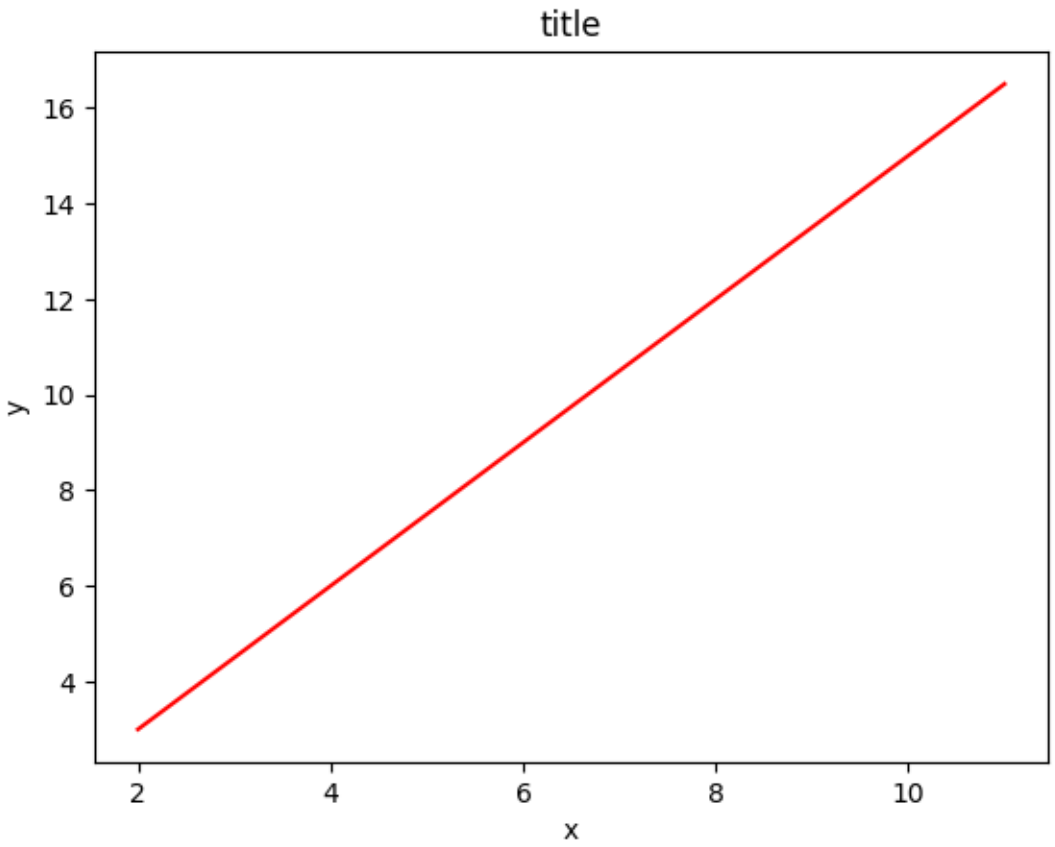
The plt.subplots() object acts as an automatic axis manager making multiple plots side by side easy.

So lets see the syntax and understand it further.

[199]:
```
# Original x and y arrays at the start of the notebook
x = np.arange(2,12)
y = 1.5*x
# Original a and b arrays at the start of the notebook
a = np.linspace(0,10,11)
b = a ** 2

# it is similar to plt.figure() except the use of tuple unpacking to grab fig␣
 ↪and axes
fig, axes = plt.subplots()

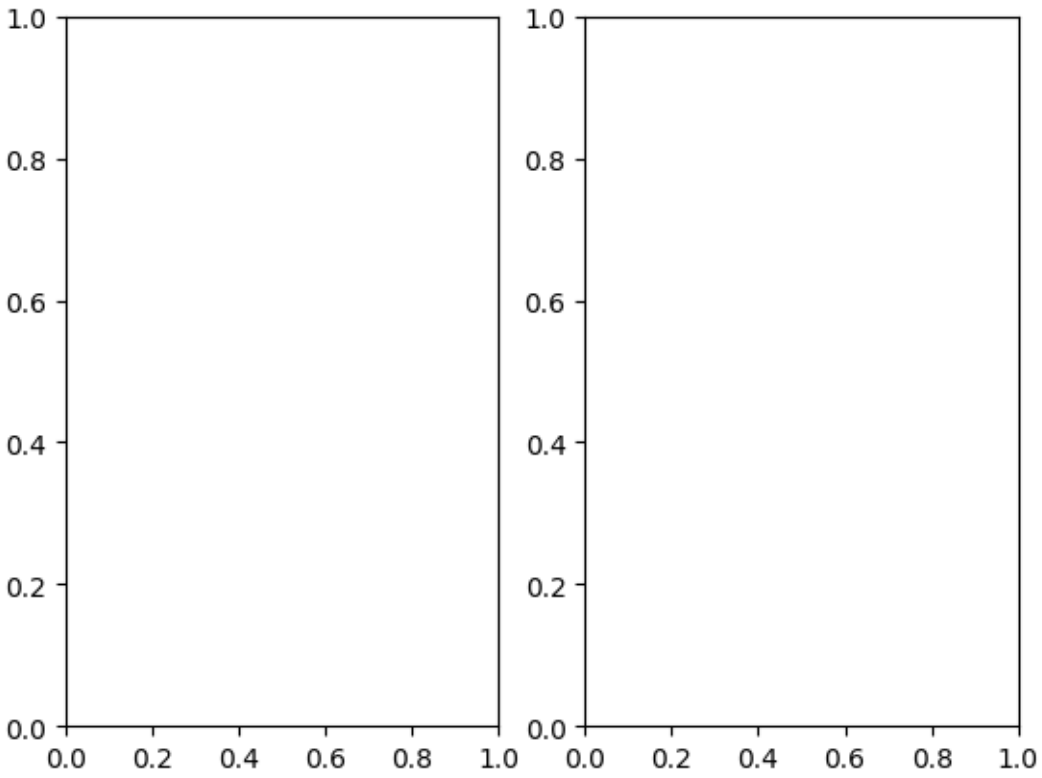
# Again use the axes object to add stuff to plot
axes.plot(x, y, 'r')
axes.set_xlabel('x')
axes.set_ylabel('y')
axes.set_title('title'); #; hides Out[]
```



0.1 Adding rows and columns

Then you can specify the number of rows and columns when creating the subplots() object:

[200]:
```
# Empty canvas of 1 by 2 subplots
fig, axes = plt.subplots(nrows=1, ncols=2)
```



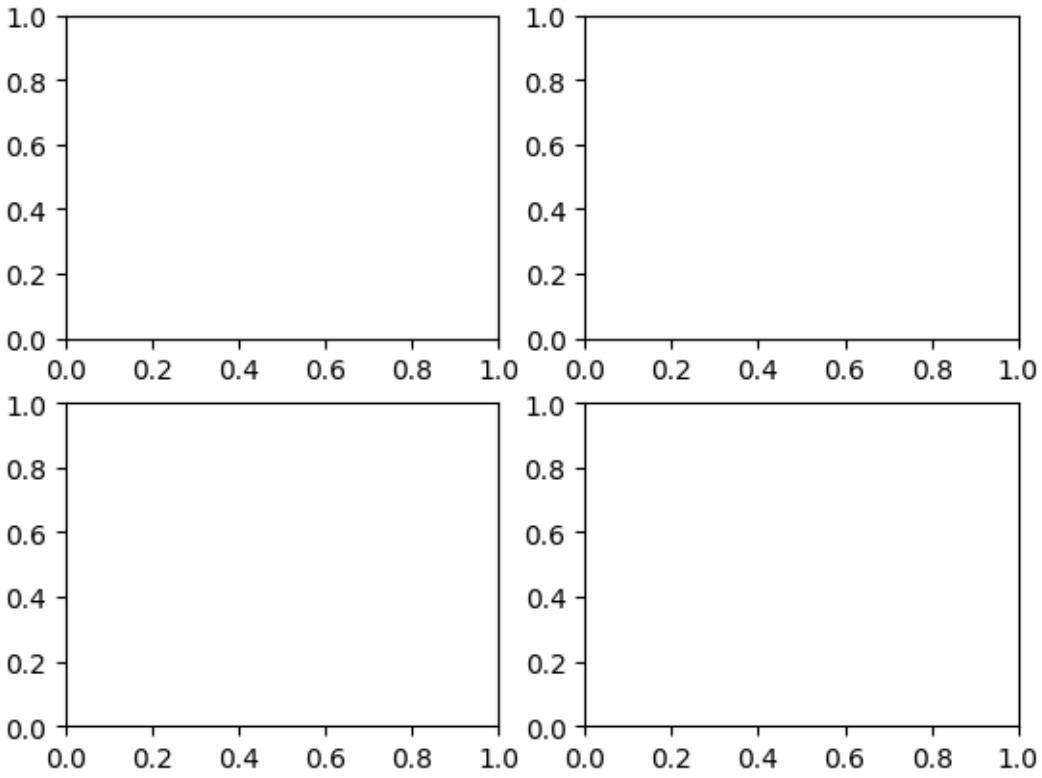
[201]:
```
# Axes is an array of axes to plot on
axes
```

[201]:
```
array([<Axes: >, <Axes: >], dtype=object)
```

[202]:
```
axes.shape
```

[202]:
```
(2,)
```

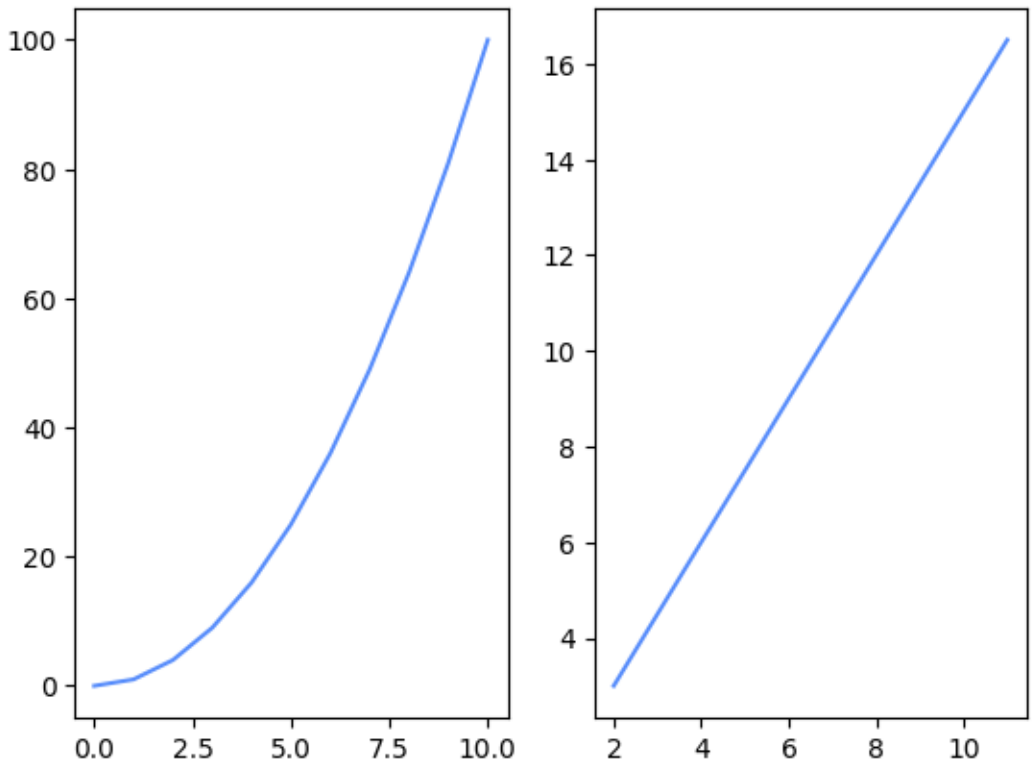
[203]:
```
# Empty canvas of 2 by 2 subplots
fig, axes = plt.subplots(nrows=2, ncols=2)
```



[204]:
```
fig,axes = plt.subplots(nrows=1,ncols=2)

axes[0].plot(a,b)
axes[1].plot(x,y)
```

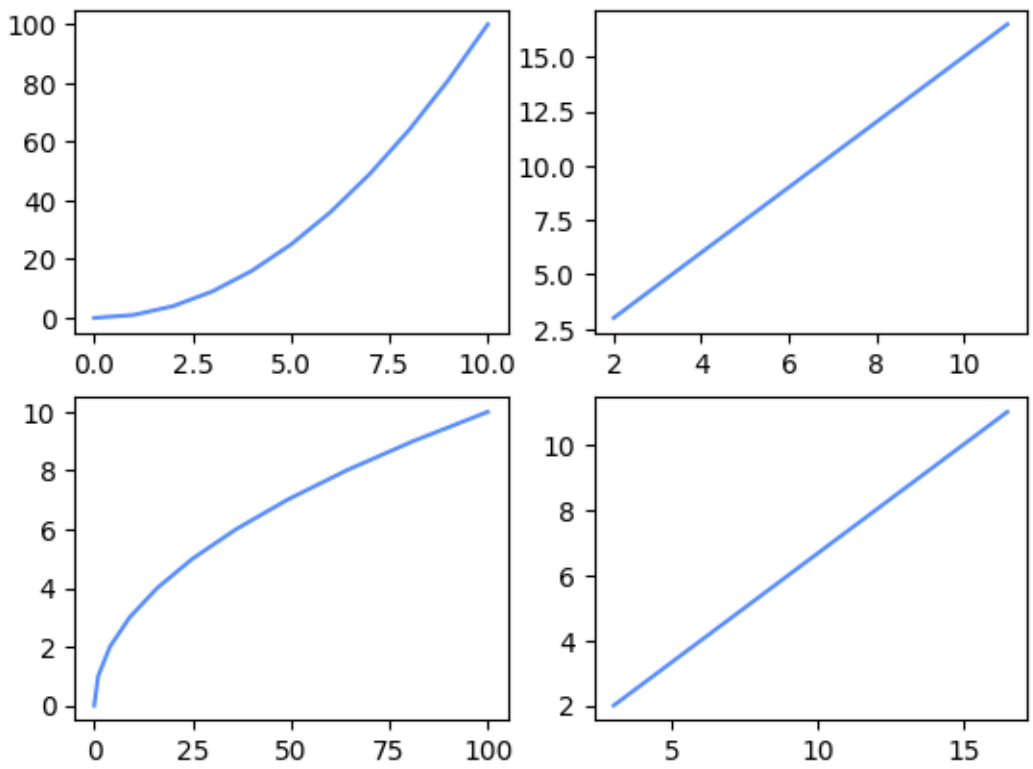
[204]:
```
[<matplotlib.lines.Line2D at 0x1368be990>]
```



[205]:
```
fig,axes = plt.subplots(nrows=2,ncols=2)


axes[0][0].plot(a,b)
axes[0][1].plot(x,y)
axes[1][0].plot(b,a)
axes[1][1].plot(y,x)
```

[205]:
```
[<matplotlib.lines.Line2D at 0x13075c440>]
```



Using the example I did before with the parametric curve, lets see how we can use subplots to create a similar figure with the main 3D plot and the two 2D projections.

Arguments	Meaning
`1, 3, 1`	1 row, 3 columns, left subplot
`1, 3, 2`	1 row, 3 columns, middle subplot
`1, 3, 3`	1 row, 3 columns, right subplot

Subplot positions start at **1**, while your Python list indices (`axes[0]`, etc.) start at **0**.

[206]:
```
fig = plt.figure(figsize=(15, 5), layout='constrained')
# add_subplot(nrows, ncols, index): rows, columns, and subplot position.
# Positions start at 1 and run left to right, then top to bottom.
# Here the layout is one row with three columns: [1] [2] [3].
axes = [
    fig.add_subplot(1, 3, 1, projection='3d'),  # Position 1 (left): 3D curve
    fig.add_subplot(1, 3, 2),  # Position 2 (middle): XY projection
    fig.add_subplot(1, 3, 3),  # Position 3 (right): YZ projection
]

# Prepare arrays x, y, z
```

```
theta = np.linspace(-4 * np.pi, 4 * np.pi, 100)
z = np.linspace(-2, 2, 100)
r = z**2 + 1
x = r * np.sin(theta)
y = r * np.cos(theta)

# Main 3D plot
axes[0].plot(x, y, z) # The label is redundant: label='Parametric curve'
axes[0].set(
    xlabel='X', ylabel='Y', zlabel='Z',
    #title='Parametric curve'
)
axes[0].set_title(
    'Parametric curve',
    color='crimson',
    fontweight='bold',
    fontsize=14,
)
#axes1.legend(loc='lower left')
axes[0].xaxis.label.set_color('orangered')
axes[0].yaxis.label.set_color('orangered')
axes[0].zaxis.label.set_color('orangered')

# 2D projection subplot
axes[1].set_facecolor('white')
axes[1].plot(x, y, color='blue')
axes[1].set(xlabel='X', ylabel='Y') #, title='XY projection')
axes[1].set_title(
    'XY projection',
    color='blue',
    fontweight='normal',
    fontsize=10,
)
axes[1].set_aspect('equal', adjustable='box')
axes[1].grid(alpha=0.3)
axes[1].tick_params(axis='both', labelcolor='blue')
axes[1].xaxis.label.set_color('blue')
axes[1].yaxis.label.set_color('blue')


# 2D projection subplot
axes[2].set_facecolor('white')
axes[2].plot(y, z, color='orange')
axes[2].set(xlabel='Y', ylabel='Z') #, title='YZ projection')
axes[2].set_title(
    'YZ projection',
    color='orange',
```

```
    fontweight='normal',
    fontsize=10
)
axes[2].set_aspect('equal', adjustable='box')
axes[2].grid(alpha=0.3)
axes[2].tick_params(axis='both', labelcolor='orange')
axes[2].xaxis.label.set_color('orange')
axes[2].yaxis.label.set_color('orange')

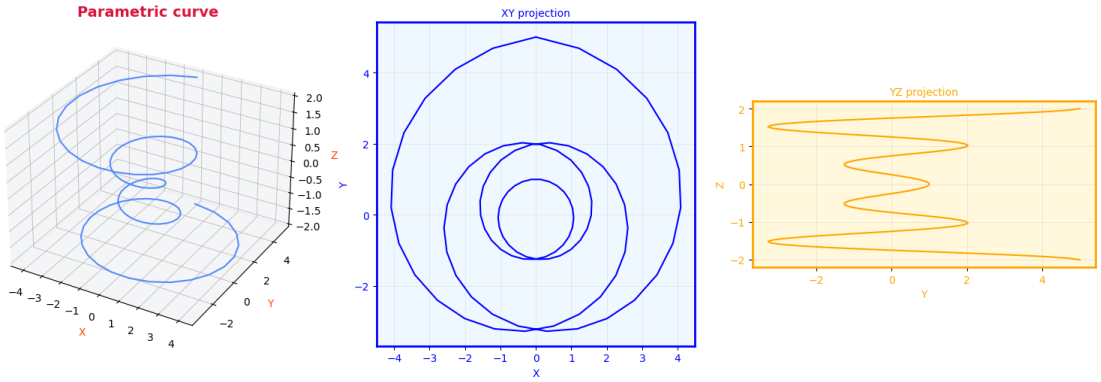
# XY projection border
for spine in axes[1].spines.values():
    spine.set_edgecolor('blue')
    spine.set_linewidth(2)

# YZ projection border
for spine in axes[2].spines.values():
    spine.set_edgecolor('orange')
    spine.set_linewidth(2)

axes[1].set_facecolor('aliceblue')
axes[2].set_facecolor('cornsilk')

plt.show()

fig.savefig('./images/week_1/figure_parametric_subplots_1.
 ↪png',bbox_inches='tight')
```



[207]:
```
fig = plt.figure(figsize=(10, 10), layout='constrained')
grid = fig.add_gridspec(2, 2)

axes = [
    fig.add_subplot(grid[0, :], projection='3d'),  # Top: both columns
    fig.add_subplot(grid[1, 0]),                   # Bottom left: XY
```

```
    fig.add_subplot(grid[1, 1]),                    # Bottom right: YZ
]

# Main 3D plot
axes[0].plot(x, y, z) # The label is redundant: label='Parametric curve'
axes[0].set(
    xlabel='X', ylabel='Y', zlabel='Z',
    #title='Parametric curve'
)
axes[0].set_title(
    'Parametric curve',
    color='crimson',
    fontweight='bold',
    fontsize=14,
)
#axes1.legend(loc='lower left')
axes[0].xaxis.label.set_color('orangered')
axes[0].yaxis.label.set_color('orangered')
axes[0].zaxis.label.set_color('orangered')

# 2D projection subplot
axes[1].set_facecolor('white')
axes[1].plot(x, y, color='blue')
axes[1].set(xlabel='X', ylabel='Y') #, title='XY projection')
axes[1].set_title(
    'XY projection',
    color='blue',
    fontweight='normal',
    fontsize=10,
)
axes[1].set_aspect('equal', adjustable='box')
axes[1].grid(alpha=0.3)
axes[1].tick_params(axis='both', labelcolor='blue')
axes[1].xaxis.label.set_color('blue')
axes[1].yaxis.label.set_color('blue')


# 2D projection subplot
axes[2].set_facecolor('white')
axes[2].plot(y, z, color='orange')
axes[2].set(xlabel='Y', ylabel='Z') #, title='YZ projection')
axes[2].set_title(
    'YZ projection',
    color='orange',
    fontweight='normal',
    fontsize=10
)
```

```
axes[2].set_aspect('equal', adjustable='box')
axes[2].grid(alpha=0.3)
axes[2].tick_params(axis='both', labelcolor='orange')
axes[2].xaxis.label.set_color('orange')
axes[2].yaxis.label.set_color('orange')

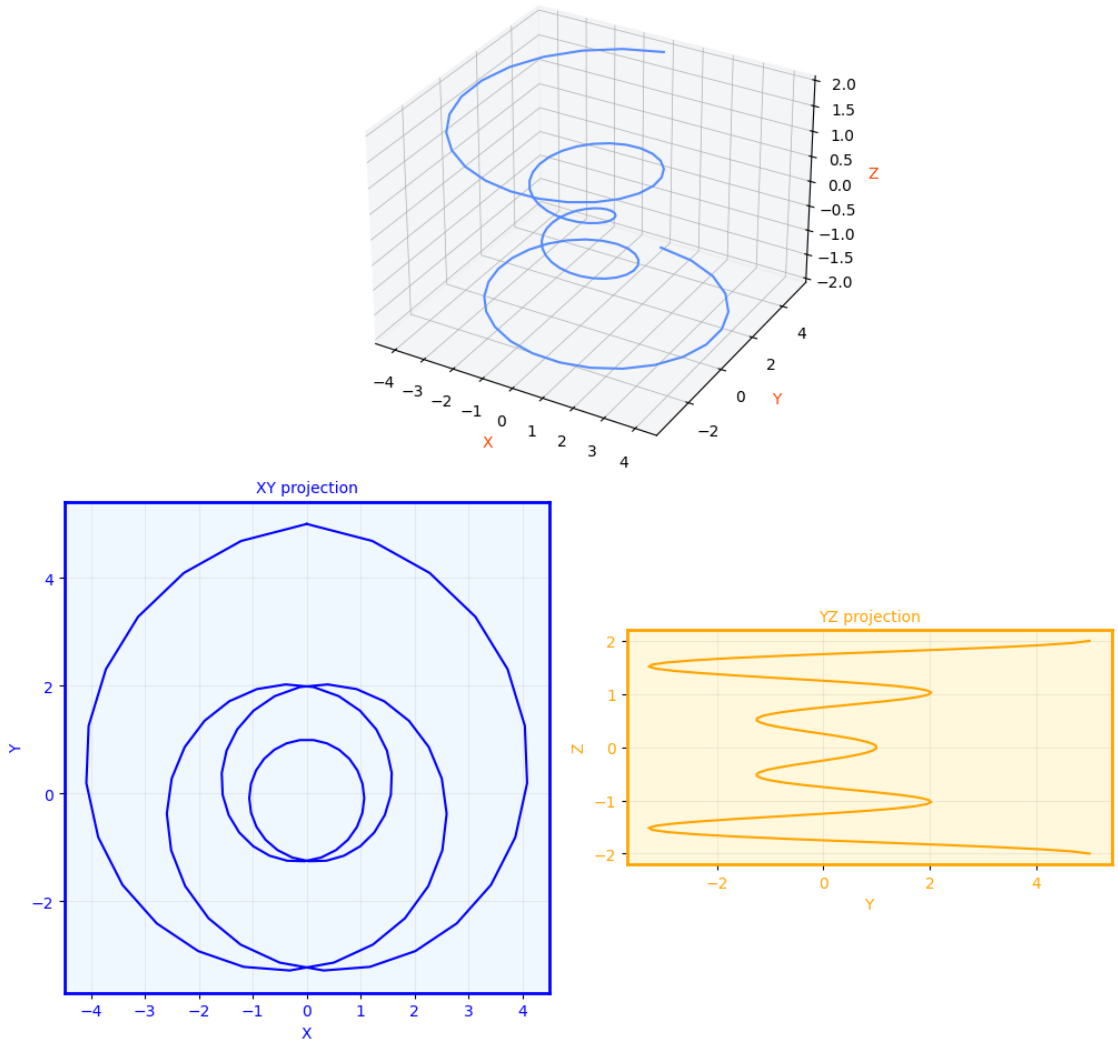
# XY projection border
for spine in axes[1].spines.values():
    spine.set_edgecolor('blue')
    spine.set_linewidth(2)

# YZ projection border
for spine in axes[2].spines.values():
    spine.set_edgecolor('orange')
    spine.set_linewidth(2)

axes[1].set_facecolor('aliceblue')
axes[2].set_facecolor('cornsilk')

plt.show()

fig.savefig('./images/week_1/figure_parametric_subplots_2.
 ↪png',bbox_inches='tight')
```


Parametric curve
X
Y
Z
XY projection
YZ projection